

LAB DIGITAL ASSIGNMENT 5

SUBJECT: NATURAL LANGUAGE PROCESSING LAB (PMDS606P)

CLASS ID - VL2025260105170

Name: Soumyadeep Ganguly

Reg No: 24MDT0082

Course: M.Sc Data Science

Problem Statement

Implement the HMM using python with any scenario/example of your choice like Weather prediction, Election polling prediction etc.

Decode with the Viterbi algorithm for HMM.

```
In [1]: import math
import numpy as np
import pandas as pd
```

```
In [2]: #handle zero probabilities
def slog(x):
    if x <= 0:
        return -math.inf
    return math.log(x)
```

```
In [3]: def viterbi(obs_seq, states, start_p, trans_p, emit_p):
    T = len(obs_seq)
    N = len(states)
    state_idx = {s:i for i,s in enumerate(states)}

    V = np.full((T, N), -math.inf)
    backpointer = np.full((T, N), -1, dtype=int)

    for i, s in enumerate(states):
        V[0, i] = slog(start_p.get(s, 0.0)) + slog(emit_p.get(s, {}).get(obs_seq[0], 0.0))
        backpointer[0, i] = -1

    for t in range(1, T):
        for j, s_j in enumerate(states):
            max_prob = -math.inf
            arg_max = -1
            emit_log = slog(emit_p.get(s_j, {}).get(obs_seq[t], 0.0))
            if emit_log == -math.inf:
                V[t, j] = -math.inf
                backpointer[t, j] = -1
                continue
            for i, s_i in enumerate(states):
                trans_log = slog(trans_p.get(s_i, {}).get(s_j, 0.0))
                if trans_log == -math.inf or V[t-1, i] == -math.inf:
                    continue
                prob = V[t-1, i] + trans_log + emit_log
                if prob > max_prob:
                    max_prob = prob
                    arg_max = i
```

```

        V[t, j] = max_prob
        backpointer[t, j] = arg_max

    last_idx = int(np.argmax(V[T-1]))
    best_log_prob = V[T-1, last_idx]
    best_path_idx = [last_idx]
    for t in range(T-1, 0, -1):
        prev = backpointer[t, best_path_idx[-1]]
        best_path_idx.append(int(prev) if prev >= 0 else -1)
    best_path_idx = list(reversed(best_path_idx))
    best_path = [states[i] if i >= 0 else None for i in best_path_idx]

    df = pd.DataFrame(V, columns=states)
    df.index.name = "time_t"
    df.insert(0, "observation", obs_seq)
    return best_path, best_log_prob, df

```

Example HMM: Weather

```

In [4]: states = ["Rainy", "Sunny"]
        observations = ["walk", "shop", "clean"]

        start_probability = {"Rainy": 0.6, "Sunny": 0.4}

        transition_probability = {
            "Rainy": {"Rainy": 0.7, "Sunny": 0.3},
            "Sunny": {"Rainy": 0.4, "Sunny": 0.6}
        }

        emission_probability = {
            "Rainy": {"walk": 0.1, "shop": 0.4, "clean": 0.5},
            "Sunny": {"walk": 0.6, "shop": 0.3, "clean": 0.1}
        }

        # Example observation sequence
        obs_seq = ["walk", "shop", "clean"]

```

```

In [5]: best_path, best_log_prob, viterbi_df = viterbi(obs_seq, states, start_probability, transition_probability, emission_probabilit

```

```
print("Observation sequence:", obs_seq)
print("Most likely state sequence (Viterbi):", best_path)
print("Log probability of that path:", best_log_prob)
print("Probability of that path (exp of log):", math.exp(best_log_prob) if best_log_prob > -700 else 0.0)
```

Observation sequence: ['walk', 'shop', 'clean']
Most likely state sequence (Viterbi): ['Sunny', 'Rainy', 'Rainy']
Log probability of that path: -4.309519943887134
Probability of that path (exp of log): 0.013439999999999999